Classification tree and Random Forest

We can apply tree-based methods to our goal of classifying observations as *hard hit* (k = 1) or *not hard hit* (k = 0). Starting with a classification tree via the `rpart` function:

```
rpart(hard_hit ~ ., data = swings, method = "class")
```

The resulting summary:

```
## Call:
## rpart(formula = hard_hit ~ ., data = swings, method = "class")
##   n= 30000
##
##   CP nsplit rel error xerror xstd
## 1  0      0         1      0    0
##
## Node number 1: 30000 observations
##   predicted class=0  expected loss=0.1630333  P(node) =1
##     class counts: 25109  4891
##    probabilities: 0.837 0.163
```

The outputed tree is just a single node with predicted class of 0. The expected classification error is then equal to the probability of k=1 in the data, 16.30%. This means that none of the variables could strongly dictate k=1 or k=0 by themselves, since `rpart()` was not able to find any splits that improved the overall purity of the tree enough to overcome the complexity parameter penalization. We can force the construction of a more complex tree by lowering the complexity parameter, which was set to the default 0.01, since the tree should grow as the complexity parameter goes to 0. Calling `rpart()` again with a significantly small cp and examining the summary shows that a cp of 0.0007156001 is sufficiently small to grow the tree. However the tree does not grow in a nice incremental way like we would hope:

```
swing_test = rpart(hard_hit ~ ., data = swings, method = "class", cp = 0.0001)
head(swing_test$cptable, 2)
```

```
##             CP nsplit rel error   xerror       xstd
## 1 0.0007556026      0 1.0000000 1.000000 0.01308143
## 2 0.0007156001     87 0.9090166 1.035984 0.01326797
```

Instead, that small value that is required to find a worthwhile split to grow the tree also allows for 86 more splits. This explosion in the tree size again shows the failure of any of the variables to stand out above the assumption of "not hard hit" when it comes to classifying observations. From the output, the cross-validation error calculated by `rpart` under `xerror` increases from the baseline branch-less classifier. This holds true in the full cp table; the estimated testing error from `rpart`'s built in 10-fold cross-validation increases monotonically with the number of splits. So a basic decision tree fails to accomplish our classification task since the leaves, or combinations of predictor relationships down the branches, it finds are not accurate to cross-validation testing data. This may be a fault of the weak recursive binary splitting algorithm being unable to identify complex relationships in the data. Or, with so many combinations of different predictors, perhaps 4891 k=1 is relatively few for the purpose of finding relationships that are true in new data. For instance, if, for whatever reason, `curveballs` in `zone 1` to `right-handed batters`, with `spin_axis < 50`, etc. for arbitrary variables are consistently `hard hit`, there may not be enough k=1 to find a true relationship after subsetting by all of those input variables.

Despite the failure of the decision tree method, we can still learn from the variable importance output. Variable importance may also be able to identify if there was a variable involved in a relationship like the one above–if that variable had, say, consistently good splits only after coming after another specific variable, and greatly improved the tree despite not being at the top of the tree:

```
head(swing_test$variable.importance,5)
```

```
##             zone        bat_speed attack_direction          plate_z
##         481.2102         450.2910         436.2525         419.6698
##     attack_angle
##         374.7943
```

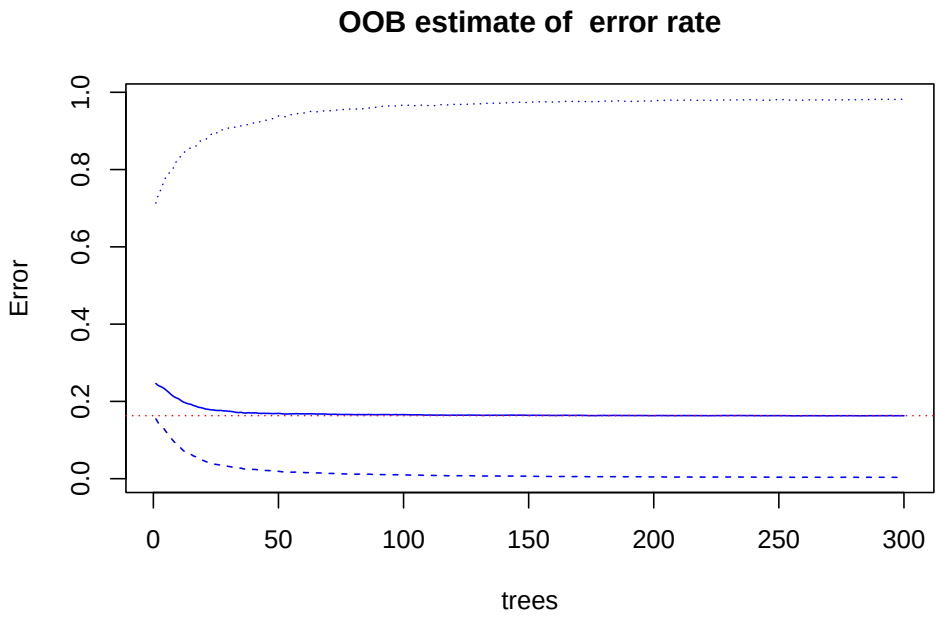
`Zone` and `plate_z` refer to the location of the pitch as it crosses the plate. At the top of the tree, they split on whether the `zone` was in 1-9 (a strike) and whether `plate_z` was below about 1.3 feet. Neither of these tell us anything that we would not have guessed already, only that the observations likely do not show consistent k=1 outside the strike zone and below 1.3 feet, since the tree was able to gain purity on these splits predicting towards k=0. The other three most important variables from the above output each correspond to swing data. That is, they contain information on the bat and swing rather than the pitched ball. Since the majority of the importance within the top five variables comes from these three, it seems likely that batting variables dictate `hard_hit` classification more strongly than pitching variables.

While the classification tree result is biased and only achieve 60 percent relative error compared to the null error rate, we attempt a random forest model for the purpose of reducing variance and overfitting. We set `mtry` equal to the square root of the number of predictors because it is the recommended default for classification, and we already know that single trees with `mtry = p` perform poorly:

```
library(randomForest)
set.seed(7)
(swing_rf = randomForest(as.factor(hard_hit) ~ ., data = swings, mtry = 6,
                         importance = TRUE, ntree = 300))
```

```
##
## Call:
##  randomForest(formula = as.factor(hard_hit) ~ ., data = swings,      mtry = 6, importance = TRUE, nt
##                Type of random forest: classification
##                      Number of trees: 300
## No. of variables tried at each split: 6
##
##         OOB estimate of  error rate: 16.28%
## Confusion matrix:
##       0  1 class.error
## 0 25025 84 0.003345414
## 1  4800 91 0.981394398
```

With 300 trees, random forest does just get below the null error rate of 16.3% represented by the dotted red line in the plot below. It does this by predicted more k=0 towards the right side of the plot to minimize the classification error at k=0. This is that reduction in overfitting we were looking for, although it comes at a huge expense of k=1 classification accuracy shown by the blue line as error. Compared t0 the null decision of k=0, the confusion matrix shows that random forest has gained 85 correct k=1 classifications and 73 incorrect k=1 classifications.



Random forest only saw a 12/30000 or 0.00004 percent improvement in error rate compared to the single tree. Variable importance is similar to that of the single tree. The three most important are all related to the batter's swing, not the pitcher or the ball. Decision trees have shown that none of the variables are individually strong at classifying hard hit observations. However, importance results from the random forest model that aggregates 500 trees, each of which chose from randomly selected subsets of predictors at each split, give us some confidence that batter-related variables (i.e. bat speed, attack direction, and attack angle) are at least *more important* for classifying hard hits than pitcher or ball-related variables.

```
head(sort(importance(swing_rf)[,4], decreasing = TRUE), 5)
```

```
##        bat_speed attack_direction     attack_angle          plate_z
##         518.1015         413.1654         401.3965         394.8506
##  swing_path_tilt
##         337.6312
```